Проектирование микропроцессорной системы на базе МК i8051

Описание программной части

Листов 28

Выполнил: Сапелин А.С.
Группа: А-12-23
Вариант: 18

Проверил: Михалин С.Н.

Москва, 2026

Подп. и дата
Инв. № дубл.
Взам. инв. №
Подп. и дата
Инв. № подл.

АННОТАЦИЯ

В данном документе приведено описание программы «Программа управления объектом», предназначенной для управления объектом микропроцессорной системой (МПС) на базе микроконтроллера AT89S53: ввода данных с клавиатуры и от внешнего устройства, формирования управляющих сигналов Y_1 и Y_2 и вывода информации на семисегментный индикатор.

Для функционирования данной программы необходима МПС, собранная по схеме электрической принципиальной варианта 18 (пояснительная записка по аппаратной части).

Исходным языком программы «Программа управления объектом» является ассемблер микроконтроллеров семейства MCS-51.

Основными функциями программы являются: инициализация МПС, сканирование клавиатуры, работа с кольцевым буфером на 1784 байт, выработка строба Y_1 длительностью 371 мс, выработка сигнала Y_2 со стробом 2452 мкс, индикация на одном разряде.

Оформление программного документа «Описание программы» произведено по требованиям ЕСПД (ГОСТ 19.101-77, ГОСТ 19.103-77, ГОСТ 19.104-78, ГОСТ 19.105-78, ГОСТ 19.106-78, ГОСТ 19.402-78, ГОСТ 19.604-78).

СОДЕРЖАНИЕ

1. Общие сведения..... 4
2. Функциональное назначение..... 5
2.1. Классы решаемых задач..... 5
2.2. Назначение программы..... 5
2.3. Сведения о функциональных ограничениях на применение..... 5
3. Описание логической структуры..... 6
3.1. Алгоритм программы..... 6
3.2. Структура программы с описанием функций составных частей и связи между ними..... 18
3.3. Связи программы с другими программами..... 19
4. Входные данные..... 20
4.1. Клавиатура..... 20
4.2. Сигнал X2..... 20
5. Выходные данные..... 21
5.1. Характер и организация выходных данных..... 21
5.2. Формат, описание и способ кодирования выходных данных..... 21
ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ..... 23
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ..... 24
ПРИЛОЖЕНИЕ А. Листинги программ..... 25

1. ОБЩИЕ СВЕДЕНИЯ

Обозначение и наименование программы — «Программа управления объектом», вариант 18, группа А-12-23.

Программа низкого уровня не требует для своей работы иных программ. Исходным языком программирования является ассемблер MCS-51 (синтаксис Intel ASM51, совместимый с Keil A51 и MCU 8051 IDE).

Исходные тексты написаны на языке ассемблера семейства 8051 в синтаксисе, поддерживаемом средствами проверки репозитория. Сборка и моделирование выполнены заданием GitHub Actions «КМ-3 программы и П32»: ассемблером asm51 и эмулятором emu51 из репозитория wwwcont/mps-kp. Версия средств проверки первого успешного прогона всех трёх программ — ревизия a55ee58b827fcd77c7e429f79f9228990fc6e44b; запуск 42074318814. Проверялись самостоятельные программы и модели устройств СХЕМА-1. Keil µVision и робот кафедры в этом прогоне не запускались; результат моделирования не заменяет проверку роботом.

2. ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ

2.1. Классы решаемых задач

- инициализация МПС;
- сканирование клавиатуры — ввод сигнала X_1 (число, введённое с клавиатуры);
- формирование сигнала Y_1 (строб заданной длительности);
- ввод сигнала X_2 (сохранение данных в кольцевом буфере);
- формирование сигнала Y_2 и сопровождающего строба;
- вывод символа на индикатор.

2.2. Назначение программы

МПС принимает информацию $\{X\}$ об объекте управления от датчиков, вырабатывает управляющие воздействия $\{Y\}$ в соответствии с заданным алгоритмом управления и подаёт их на исполнительные механизмы. Программа разбита на подзадачи по функциональному признаку:

- инициализация МПС;
- обработка прерываний: опрос клавиатуры, запись в буфер, таймеры генераторов сигналов Y;
- определение кода нажатой клавиши;
- вывод данных на семисегментный индикатор;
- формирование сигнала Y_2;
- обеспечение работы кольцевого буфера (запись и чтение отсчётов X_2).

В соответствии с вариантом (табл. 1 ТЗ) разработан код трёх программ:

- программа 1 — обработчик прерывания клавиатуры (сканирование матрицы), код нажатой клавиши в аккумуляторе или 0FFh;
- программа 2 — чтение из кольцевого буфера (данные через аккумулятор) с управлением флагами пустоты и переполнения;
- программа 3 — вывод символа по коду на индикатор с гашением через T_3 (прерывание таймера).

2.3. Сведения о функциональных ограничениях на применение

В качестве основы для построения МПС выбран микроконтроллер семейства MCS-51 — AT89S53, тактовая частота 12 МГц (машинный цикл 1 мкс). Таймер 2 и сторожевой таймер не используются (совместимость с i8051).

Интерфейсы:

- входные данные X_2 — 8-разрядное целое без знака по независимой параллельной шине со стробом записи низкого уровня в двухпортовое ОЗУ, нерегулярно, до 4 отсчётов в секунду;
- входные данные X_1 — десятичная цифра 0–9 с матричной клавиатуры 4×3 (сканирование по столбцам);
- выходной сигнал Y_2 — 8-разрядный параллельный код в выходном регистре со стробом низкого уровня длительностью 2452 мкс на выводе P1.5;
- выходной сигнал Y_1 — строб низкого уровня длительностью 371 мс на выводе P1.4;
- индикация — один семисегментный индикатор (общий анод), время индикации символа не более T_3 = 3,240 с.

3. ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ

3.1. Алгоритм программы

Работу полной МПС я разделил на основной цикл и обработчики прерываний. Инициализация задаёт SP = 07h, пустой буфер, X1 = 0, погашенный индикатор, режимы таймеров и исходные уровни стробов (рисунок 3.1). После неё выполняется основной цикл (рисунок 3.2). Если X1 отличается от нуля, в буфере есть отсчёт и предыдущий строб Y2 закончен, цикл читает самый старый отсчёт и выдаёт новое значение Y2 со стробом. Если X1 = 0 или буфер пуст, цикл не читает данные и при необходимости записывает Y2 = 0. Запись в выходной регистр выполняется только после окончания предыдущего строба: иначе изменились бы данные, которые ещё принимает внешнее устройство.

Клавиатура обслуживается по INT0, поступление X2 — по INT1. Timer0 отсчитывает интервалы T1 и T3, Timer1 завершает строб Y2 длительностью T2. Приём X2 и сопровождение управляющего выхода получают высокий приоритет. Схемы описывают алгоритмы всей МПС, а три самостоятельные программы реализуют только задания варианта 18: сканирование клавиатуры, чтение буфера и индикацию. Поэтому остановка jmp $ после проверяемой процедуры в самостоятельном файле не заменяет основной цикл полной системы.

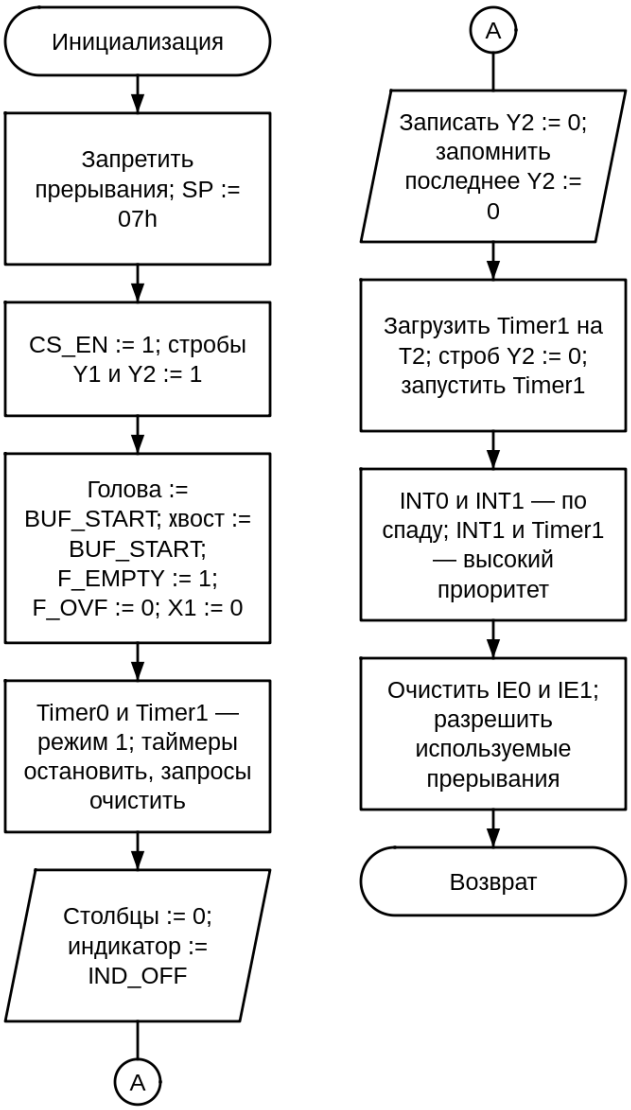


Рис. 3.1. Схема алгоритма: Инициализация МПС

При включении я сначала запрещаю прерывания, чтобы обработчик не использовал ещё не подготовленные переменные. Указатели головы и хвоста получают BUF_START,

устанавливается F_EMPTY, а F_OVF сбрасывается. После настройки таймеров и вывода начальных состояний разрешаются нужные прерывания. Процедура возвращается в основную программу; ожидание jmp $ предусмотрено только в трёх самостоятельных файлах для проверки.

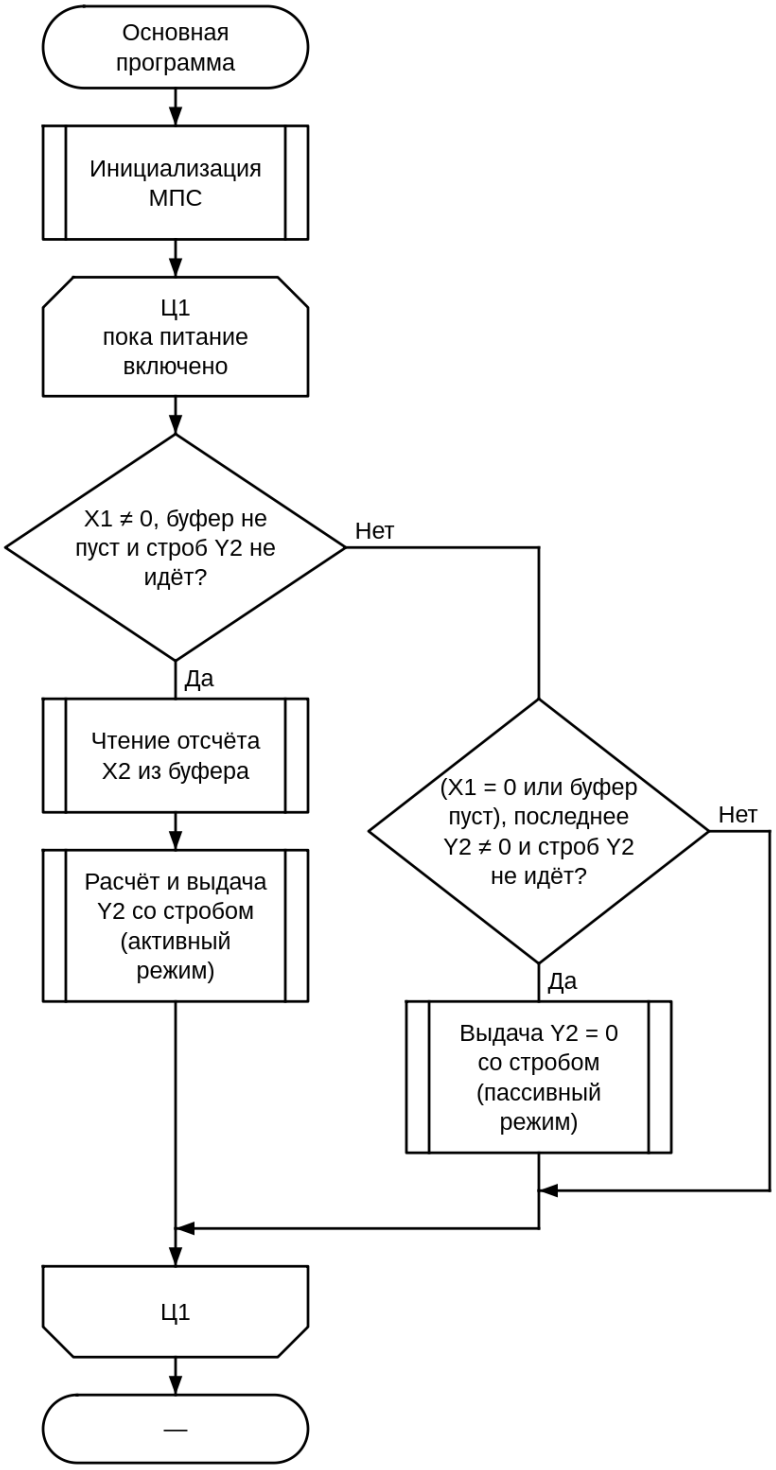


Рис. 3.2. Схема алгоритма: Основной цикл программы

Основной цикл обрабатывает отсчёты по очереди и проверяет три условия перед чтением: X1 не равен нулю, буфер не пуст, строб Y2 завершён. При выполнении условий читается один отсчёт, затем формируется управляющий байт. Пассивная ветвь нужна и при X1 = 0, и при отсутствии данных: в обоих случаях выход должен перейти в ноль. Если ноль уже записан, повторная запись не требуется; если ещё идёт строб, цикл ждёт его окончания без

изменения регистра. Это исправление внесено только в мою схему алгоритма по требованиям раздела 1 ТЗ-2026.

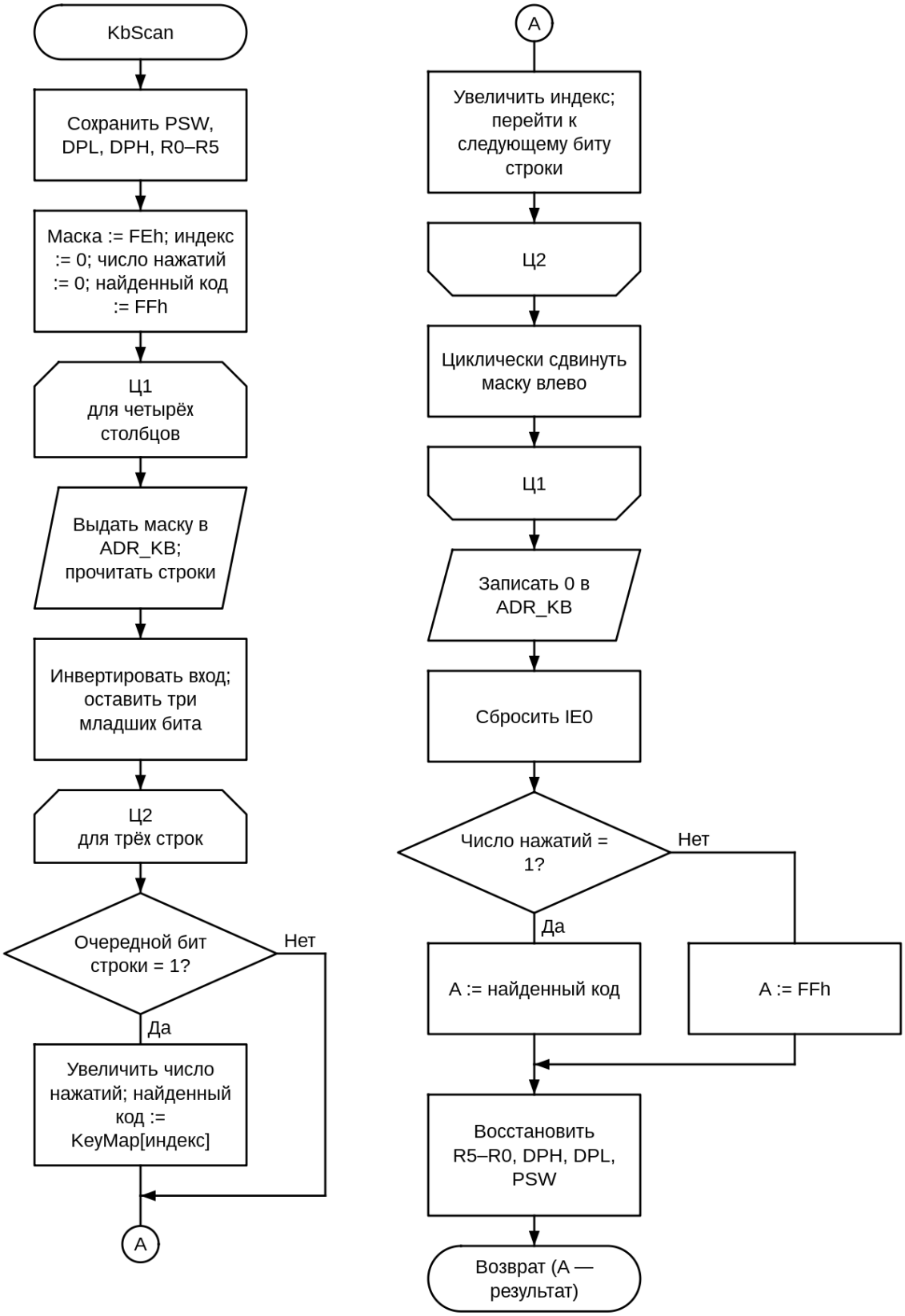


Рис. 3.3. Схема алгоритма: Опрос клавиатуры (процедура KbScan программы 1)

Процедура KbScan опускает по одному столбцу и проверяет все три строки каждого столбца. После инверсии прочитанного байта маска удаляет биты, не относящиеся к строкам клавиатуры. Считаются все замкнутые контакты, поэтому две клавиши в разных столбцах

также приводят к FFh. Коды выбираются из KeyMap в порядке обхода столбцов: 1, 4, 7, 2, 5, 8, 3, 6, 9, 10, 11, 0. Перед возвратом все столбцы переводятся в ноль, очищается IE0 и восстанавливаются сохранённые регистры; результат остаётся в A.

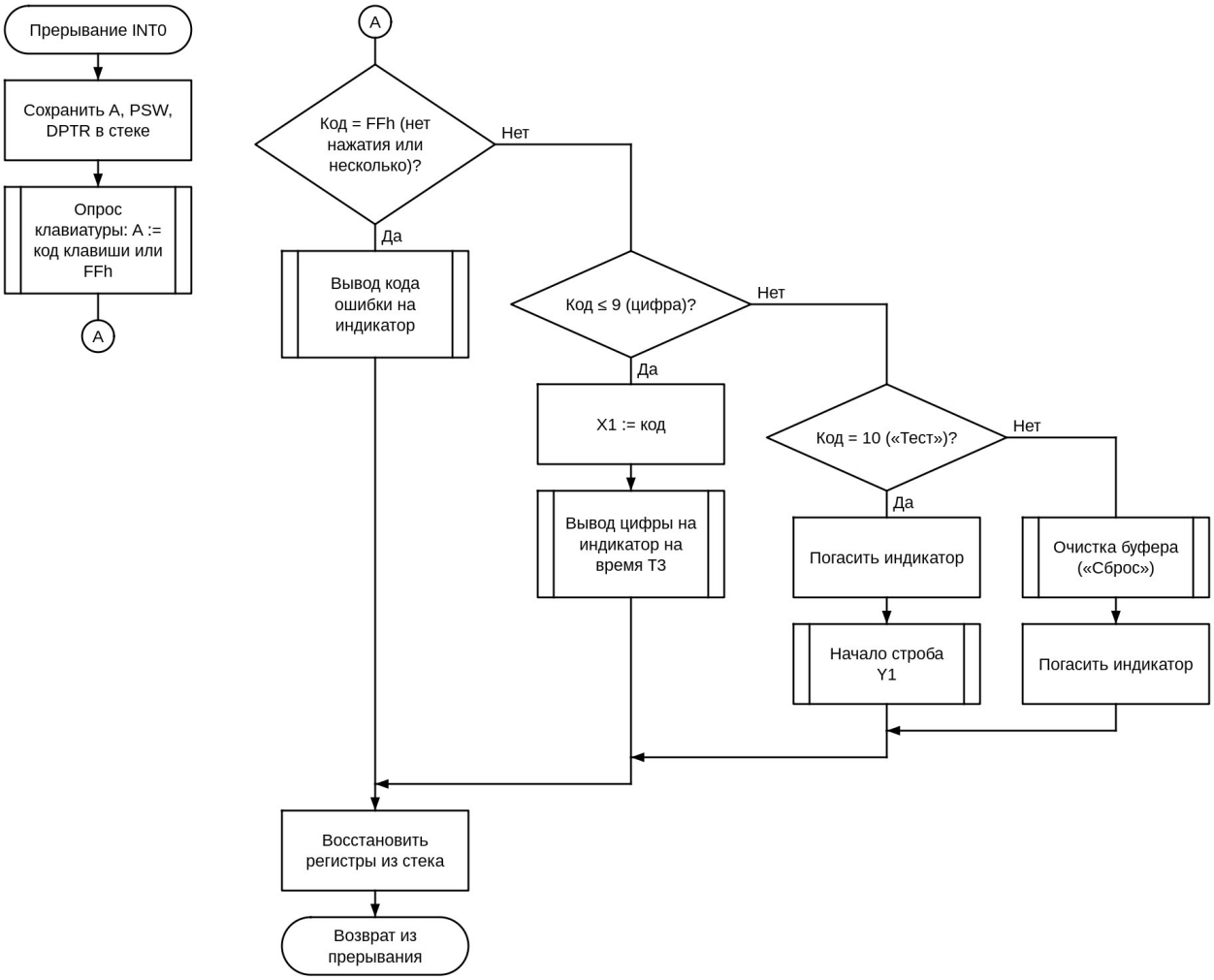


Рис. 3.4. Схема алгоритма: Обработчик прерывания INT0: реакция на клавишу

В полной системе обработчик INT0 использует результат опроса для выбора действия. Цифра изменяет X1 и выводится на индикатор на время T3; «Тест» запускает Y1 и гасит индикатор, «Сброс» очищает буфер и также гасит индикатор. Значение FFh означает, что единственная клавиша не определена, и не должно изменять X1. Эта схема описывает реакцию полной МПС; самостоятельная программа 1 заканчивает обработку возвратом кода в A и не реализует действия оператора.

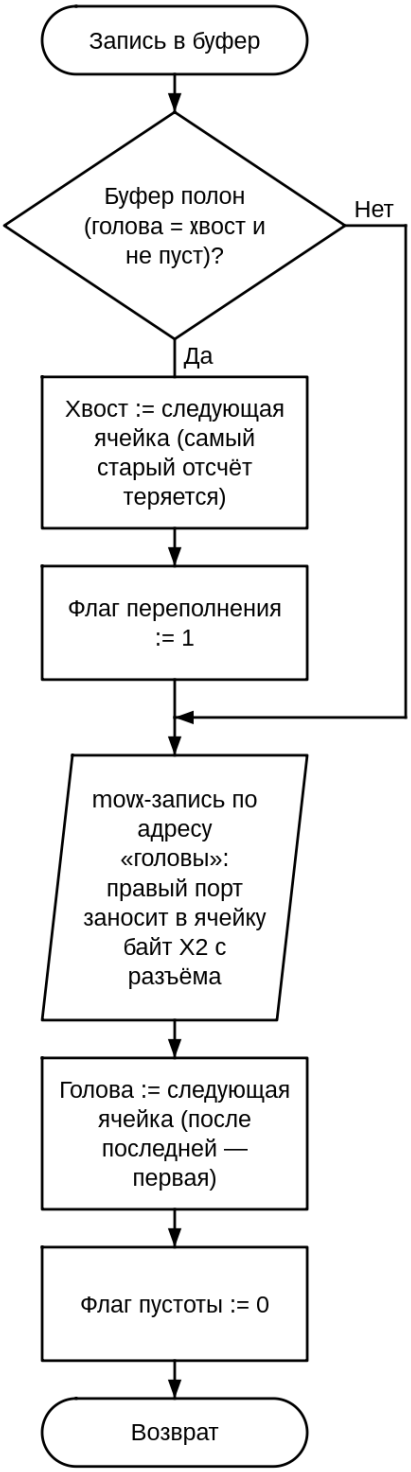


Рис. 3.5. Схема алгоритма: Запись отсчёта в кольцевой буфер

Голова указывает на место следующей записи, хвост — на самый старый непрочитанный отсчёт. Для полной системы выбрана политика хранения V отсчётов: совпадение указателей при F_EMPTY = 0 означает полный буфер. Новая запись в таком состоянии вытесняет самый старый отсчёт и сдвигает хвост. После записи голова переходит к следующей ячейке с заворотом на BUF_START, а флаг пустоты снимается. Эта операция в составе МПС использует правый порт IDT7005 и байт X2 с внешнего разъёма.

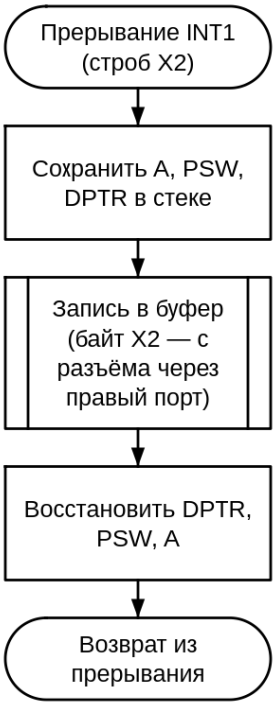


Рис. 3.6. Схема алгоритма: Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства

Строб внешнего устройства вызывает INT1. Обработчик сохраняет контекст и обращается к процедуре записи по адресу головы. В СХЕМА-1 адреса обоих портов IDT7005 задаёт МК, а команда movx-записи создаёт сигнал WR для правого порта: сохраняется X2 с разъёма, а не содержимое A. После обновления указателей и флагов контекст восстанавливается и выполняется возврат из прерывания.

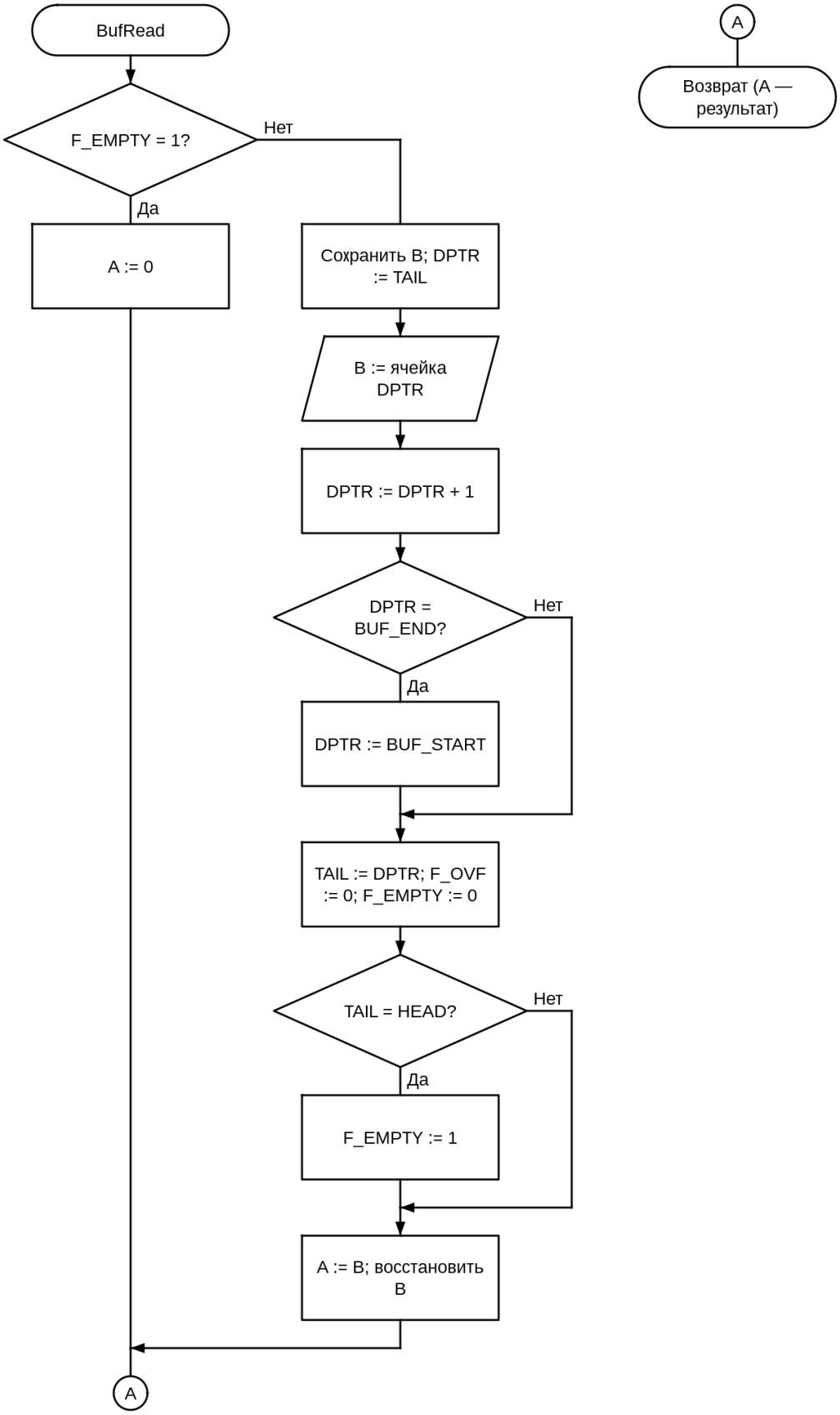


Рис. 3.7. Схема алгоритма: Чтение отсчёта из кольцевого буфера (BufRead)

BufRead сначала проверяет F_EMPTY. При пустом буфере процедура возвращает A = 0, не обращается к внешнему ОЗУ и сохраняет указатели и флаги. В остальных случаях байт читается по хвосту и временно хранится в B, пока вычисляется новый адрес. BUF_END обозначает первую ячейку за кольцом, поэтому достижение этого адреса приводит к переходу

на BUF_START. После чтения снимается F_OVF, а F_EMPTY устанавливается, если хвост догнал голову; прежний B восстанавливается, прочитанный байт возвращается в A.



Рис. 3.8. Схема алгоритма: Очистка кольцевого буфера (клавиша «Сброс»)

Очистка меняет служебное состояние кольца, поэтому перезаписывать все 1784 ячейки нулями не требуется. На время изменения указателей запрещается INT1: иначе новый отсчёт мог бы изменить голову между операциями очистки. Хвост приравнивается к голове, устанавливается пустота и снимается переполнение. После разрешения приёма следующий отсчёт снова считается первым действительным элементом.

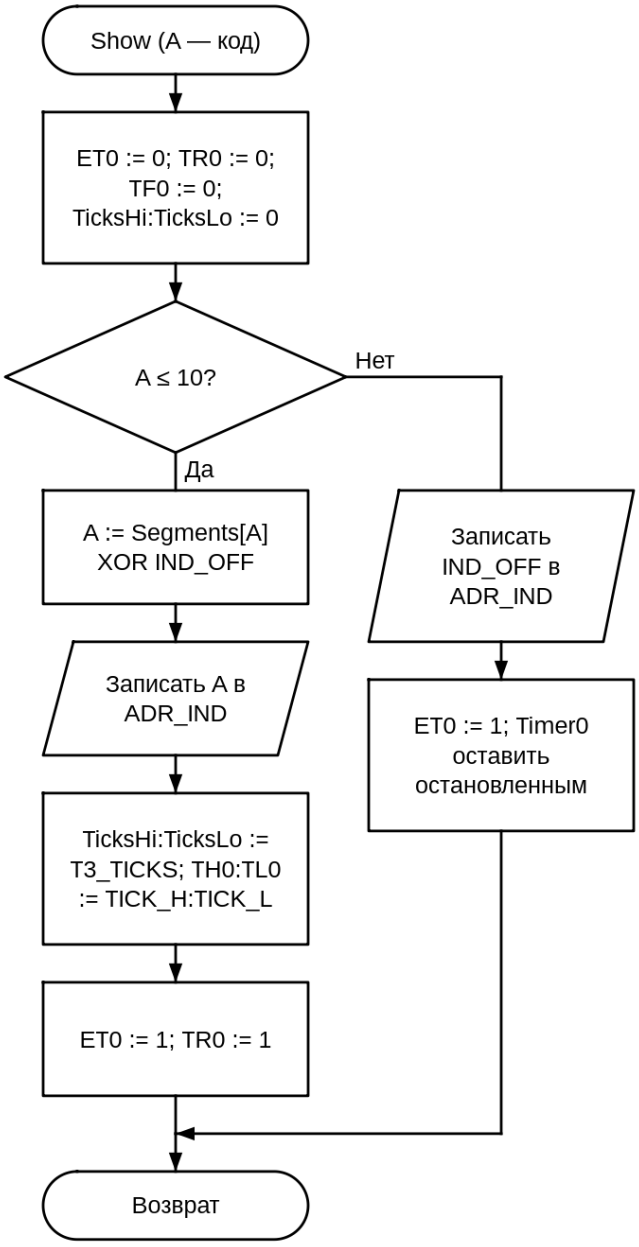


Рис. 3.9. Схема алгоритма: Вывод символа на индикатор (Show программы 3)

В самостоятельной программе 3 процедура Show сначала запрещает ET0, останавливает Timer0 и отменяет прежний отсчёт. Для кода от 0 до 10 из Segments выбирается изображение цифры или E; XOR с IND_OFF преобразует таблицу для общего анода. Регистр индикатора удерживает полученный байт, пока процедура загружает счётчик T3_TICKS и запускает Timer0. Коды от 11 до 255 сразу гасят индикатор и оставляют таймер остановленным. Новый допустимый символ запускает отсчёт заново, процедура не ждёт истечения T3.

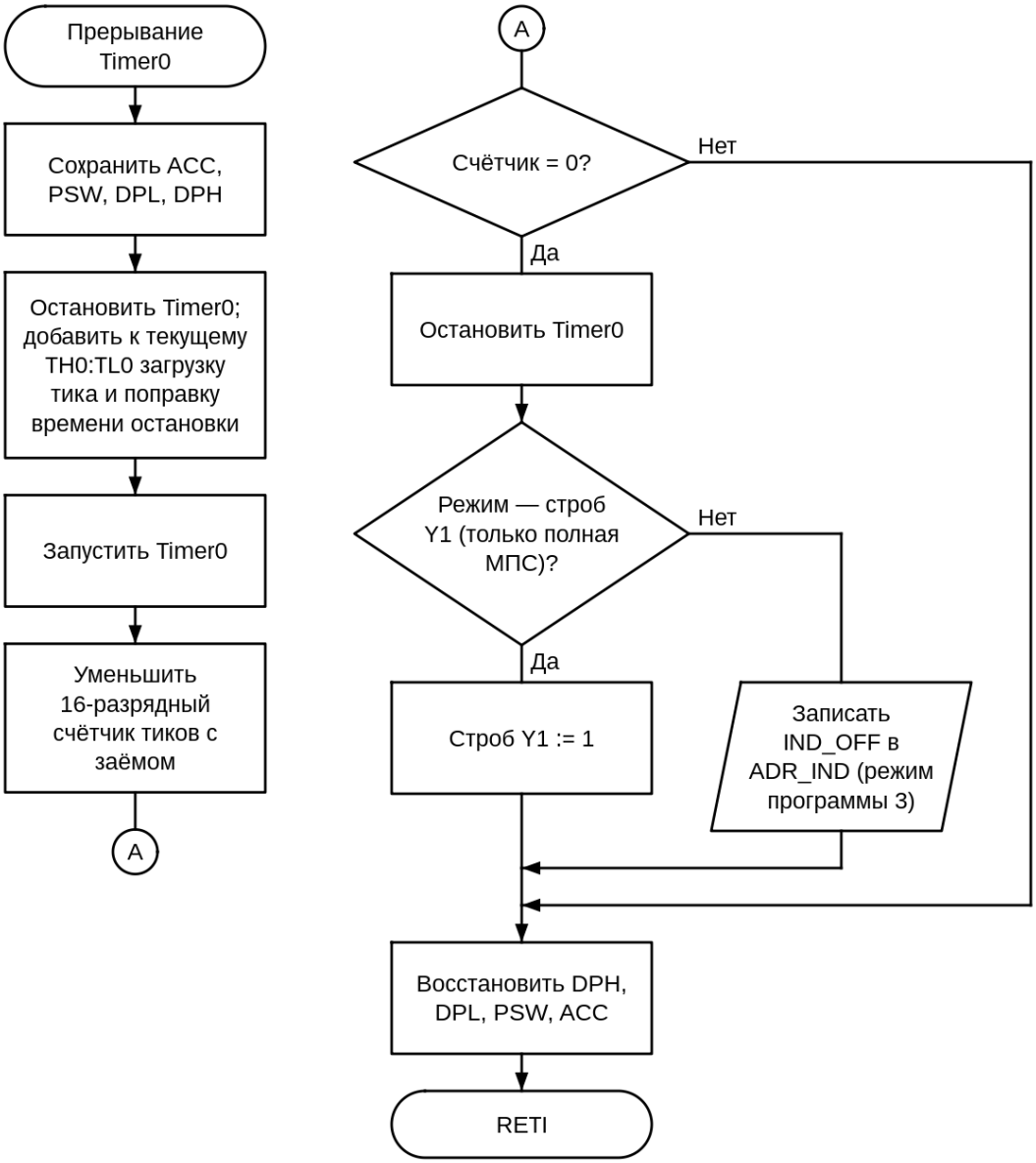


Рис. 3.10. Схема алгоритма: Обработчик прерывания Timer0: отсчёт T1 и T3

В алгоритме полной системы Timer0 используется для T1 и T3; назначение текущего отсчёта задаётся режимом. В самостоятельной программе 3 используется только режим индикации и счётчик TicksHi:TicksLo. Обработчик T0Isr сохраняет ACC, PSW, DPL и DPH, а при перезагрузке учитывает уже прошедшие циклы и семь циклов остановленного таймера. После каждого тика счётчик уменьшается с заёмом между байтами. При нуле таймер останавливается и индикатор гасится; иначе выполняется восстановление контекста и RETI.

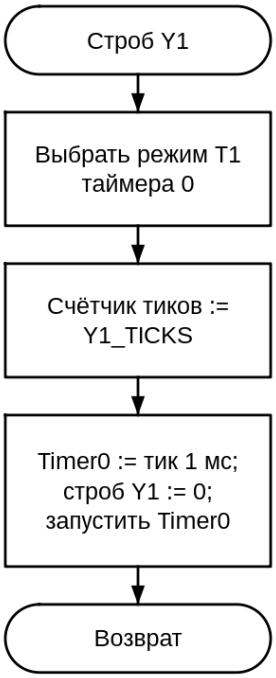


Рис. 3.11. Схема алгоритма: Начало строба Y1

Формирование Y1 предусмотрено в алгоритме полной системы. Процедура выбирает режим T1 таймера 0, загружает число тиков Y1_TICKS и переводит P1.4 в активный ноль. После запуска таймера она возвращается, поэтому обработка других событий продолжается. Конец интервала формирует обработчик Timer0, возвращая P1.4 в единицу.

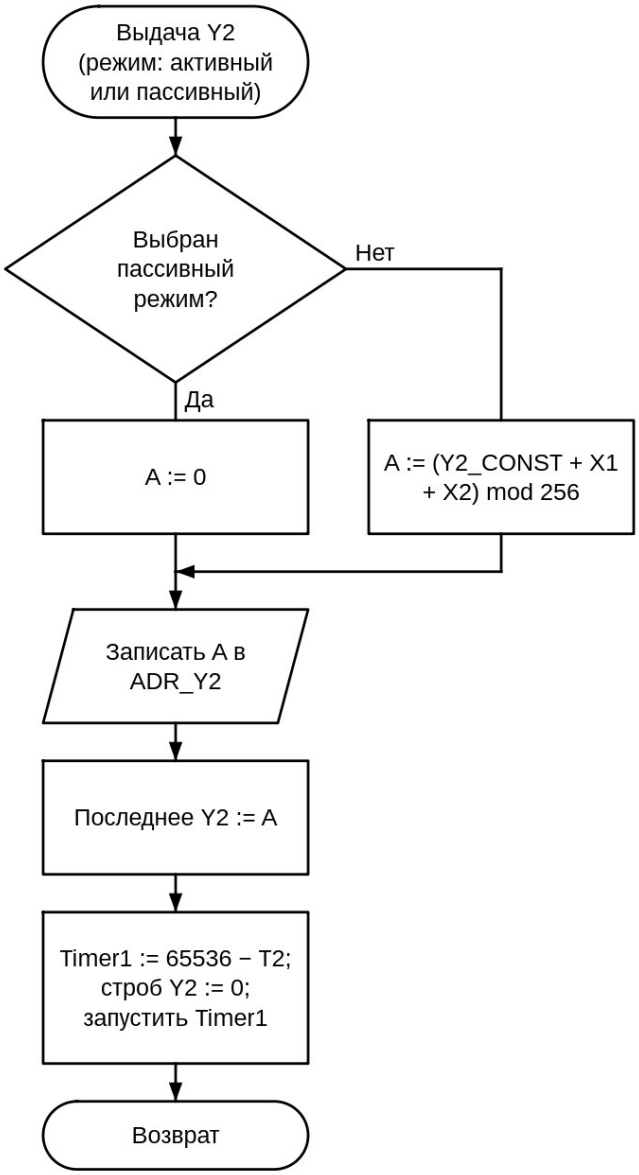


Рис. 3.12. Схема алгоритма: Расчёт и выдача сигнала Y2

Активная выдача вызывается после чтения действительного отсчёта: Y2 вычисляется как (Y2_CONST + X1 + X2) mod 256. Признак активной выдачи сохраняется отдельно от F_EMPTY, поскольку чтение последнего отсчёта уже устанавливает флаг пустоты, но этот отсчёт ещё должен быть обработан. Для пассивной выдачи используется ноль без чтения буфера. Сначала байт записывается в ADR_Y2 и запоминается как последнее состояние выхода, затем P1.5 переводится в ноль и запускается Timer1. Так данные в регистре готовы к началу строба и сохраняются до следующей разрешённой записи.



Рис. 3.13. Схема алгоритма: Обработчик прерывания Timer1: конец строба Y2

Timer1 обслуживает только интервал T2. Его обработчик останавливает таймер и возвращает строб на P1.5 в единицу. Основной цикл определяет по этому состоянию, что можно записывать следующий управляющий байт. Значение регистра Y2 при завершении строба сохраняется: T3 требует удерживать его до новой записи.

3.2. Структура программы с описанием функций составных частей и связи между ними

Программа состоит из инициализации, основной программы (ожидание прерываний) и подпрограмм, описанных ниже. Описание каждой подпрограммы взято из заголовочного комментария в исходном тексте (приложение А).

3.2.1 Подпрограмма KbScan (программа 1)

Назначение: KbScan проверяет все контакты матрицы и считает нажатые клавиши.

Входные данные: строки и столбцы клавиатуры, подключённой к ADR_KB.

Выходные данные: A — код 0–11 единственной клавиши либо 0FFh при ином числе нажатий.

Используемые регистры: A; остальные используемые регистры сохраняются.

3.2.2 Подпрограмма BufRead (программа 2)

Назначение: BufRead извлекает самый старый отсчёт из кольцевого буфера.

Входные данные: HEAD, TAIL и флаги из vars.inc; данные во внешнем ОЗУ.

Выходные данные: после чтения TAIL передвинут, F_OVF = 0, F_EMPTY отражает опустошение.

Используемые регистры: A, DPTR, флаги PSW; B сохраняется.

3.2.3 Подпрограмма Show (программа 3)

Назначение: Show выводит символ с автоматическим гашением через T3.

Входные данные: A — код 0–9 или 10 для E; остальные коды гасят индикатор.

Выходные данные: символ в регистре ADR_IND, заново запущенный отсчёт T3 либо остановка.

Используемые регистры: A, DPTR, PSW; обновляет TicksLo, TicksHi и Timer0.

3.2.4 Обработчик прерывания Timer0 — T0Isr (программа 3)

Назначение: T0Isr отсчитывает тики и выключает индикатор в конце T3.

Входные данные: TicksLo:TicksHi и текущее состояние Timer0.

Выходные данные: счётчик уменьшен; при нуле Timer0 остановлен и индикатор погашен.

Используемые регистры: Timer0 и счётчик; A, PSW и DPTR восстанавливаются перед RETI.

Распределение ресурсов микроконтроллера — прерывания, адреса внешних устройств и переменные — приведено в таблицах 3.1–3.3.

Таблица 3.1

Прерывания

Источник	**Вектор**	**Назначение**	**Запуск**
INT0 (P3.2)	0003h	нажатие клавиши	по спаду
Timer0	000Bh	отсчёт T_1 (371 мс) и T_3 (3240 мс), тик 1 мс	переполнение
INT1 (P3.3)	0013h	строб записи X_2	по спаду
Timer1	001Bh	отсчёт T_2 (2452 мкс)	переполнение
UART	0023h	не используется (заглушка)	—

Таблица 3.2

Адреса внешних устройств (обращение командами movx при P3.4 = 1)

Устройство	**Выход дешифратора**	**Адреса**
Индикатор	Y7	0E000h–0FFFFh
Клавиатура	Y1	2000h–3FFFh
Буфер (IDT7005)	Y5	0A000h–0BFFFh
Регистр Y2	Y6	0C000h–0DFFFh

Таблица 3.3

Переменные во внутреннем ОЗУ

Имя	**Адрес**	**Назначение**
HEAD_L, HEAD_H	47h, 48h	«голова» — адрес ячейки буфера для следующей записи
TAIL_L, TAIL_H	49h, 4Ah	«хвост» — адрес ячейки буфера для следующего чтения
F_EMPTY	22h.2 (бит 12h)	флаг пустоты буфера
F_OVF	23h.6 (бит 1Eh)	флаг переполнения буфера
TicksLo	30h	переменная программы 3
TicksHi	31h	переменная программы 3

3.3. Связи программы с другими программами

Программа «Программа управления объектом» в ходе своей работы не использует других программ.

4. ВХОДНЫЕ ДАННЫЕ

4.1. Клавиатура

Клавиатура — матрица 4×3 (4 столбцов × 3 строк), сканирование по столбцам: в регистр столбцов (адрес 2000h, разряды D0…D3) выдаётся 0 в опрашиваемый столбец, строки читаются через повторитель по тому же адресу (разряды D0…D2, 0 — контакт замкнут). Нажатие любой клавиши при всех столбцах в 0 вызывает прерывание INT0. Коды клавиш (табл. 3 ТЗ) и их расположение — в таблице 4.1.

Таблица 4.1

Расположение клавиш и их коды («клавиша» — код)

	Col1	**Col2**	**Col3**	**Col4**
Row1	«1» — 1	«2» — 2	«3» — 3	«Т» — 10
Row2	«4» — 4	«5» — 5	«6» — 6	«С» — 11
Row3	«7» — 7	«8» — 8	«9» — 9	«0» — 0

Результат опроса — код единственной нажатой клавиши (0–9 — цифры, 10 — «Тест», 11 — «Сброс») или 0FFh, если не нажато ничего или нажато больше одной клавиши. Цифра становится значением X_1 (начальное значение $X_1 = 0$).

4.2. Сигнал X2

X_2 — 8-разрядное целое без знака. Внешнее устройство выставляет отсчёт на разъёме доступа к буферу и сопровождает его стробом, который вызывает прерывание INT1. Буфером управляет МК: обработчик делает movx-запись по адресу «головы», и правый порт двухпортового ОЗУ IDT7005 заносит в ячейку байт X_2 с разъёма. Отсчёты хранятся в кольцевом буфере на 1784 байт (адреса 0A000h–0A6F7h).

5. ВЫХОДНЫЕ ДАННЫЕ

5.1. Характер и организация выходных данных

Индикация. На индикаторе отображается последняя введённая цифра — до ввода следующей, но не дольше T_3 = 3,240 с; при ошибке — код ошибки с горящей точкой dp. Начальное состояние — индикатор погашен.

Кольцевой буфер во внешнем ОЗУ. Ячейки 0A000h–0A6F7h. «Голова» — адрес ячейки для следующей записи, «хвост» — для следующего чтения; после последней ячейки следует первая. Буфер пуст — флаг пустоты = 1 (голова = хвост). Если буфер заполнен, новый отсчёт затирает самый старый (хвост сдвигается) и устанавливается флаг переполнения; после чтения флаг переполнения сбрасывается.

Управляющие сигналы. Y_1 — строб низкого уровня длительностью 371 мс по клавише «Тест». $Y_2 = (12 + 18 + X_1 + X_2) \bmod 256$ при $X_1 \neq 0$, иначе $Y_2 = 0$ (пассивное состояние); вырабатывается, пока в буфере есть данные и $X_1 \neq 0$. Каждая запись Y_2 в выходной регистр сопровождается стробом длительностью 2452 мкс; до записи регистр выдаёт предыдущее значение.

Расчётные параметры работы программы. Константы таймеров (кварц 12 МГц, машинный цикл $T_{мц}$ = 12 / f = 1 мкс, режим 1 — 16 бит):

TH1:TL1 = 65536 − T2 / T_мц = 65536 − 2452 = 63084 = 0F66Ch (строб Y2, таймер 1)

TH0:TL0 = 65536 − 1 мс / T_мц = 65536 − 1000 = 64536 = 0FC18h (тик 1 мс, таймер 0)

N_T1 = T1 / 1 мс = 371; N_T3 = T3 / 1 мс = 3240 (число тиков)

Минимальное время заполнения буфера (при $X_1 = 0$ сигнал Y_2 пассивен и буфер не читается, отсчёты поступают с максимальной интенсивностью):

t_min = V / f_X2 = 1784 / 4 = 446,0 с (7,4 мин).

Максимальная глубина стека для программы 1 — 13 байт (SP от 07h до 14h). Значение получено прогоном программы в симуляторе на всех сценариях проверки (каждая клавиша, одновременные нажатия, помеха на INT0) и складывается из адреса возврата при входе в обработчик INT0 (2 байта), адреса возврата при вызове процедуры опроса (2 байта) и сохраняемых в стеке регистров.

5.2. Формат, описание и способ кодирования выходных данных

Код символа (табл. 2 ТЗ) преобразуется в код сегментов по таблице 5.1: разряды D0…D7 регистра индикатора соответствуют сегментам a, b, c, d, e, f, g, dp; индикатор с общим анодом — сегмент горит при 0.

Таблица 5.1

Коды сегментов индикатора (dp g f e d c b a)

Код	**Символ**	**Двоичный**	**Шестн.**
0	«0»	11000000	0C0h
1	«1»	11111001	0F9h
2	«2»	10100100	0A4h
3	«3»	10110000	0B0h
4	«4»	10011001	99h
5	«5»	10010010	92h
6	«6»	10000010	82h
7	«7»	11111000	0F8h
8	«8»	10000000	80h
9	«9»	10010000	90h
10	«E»	10000110	86h
11 и больше	погашен	11111111	0FFh

Таблица 5.2

Коды ошибок (символ с горящей точкой dp, индикация до нажатия любой клавиши, но не дольше T3)

Индикация	**Смысл**	**Когда выставляется**
«1.»	нажато несколько клавиш	опрос клавиатуры нашёл больше одного замкнутого контакта
«2.»	нажатие не распознано	прерывание пришло, а замкнутых контактов нет
«3.»	переполнение буфера	запись X_2 затёрла непрочитанный отсчёт
«4.»	повторный «Тест»	«Тест» нажат, пока идёт строб Y_1

Таблица задаёт возможные диагностические сообщения полной МПС: «1.» — несколько клавиш, «2.» — нераспознанное нажатие, «3.» — вытеснение непрочитанного отсчёта, «4.» — повторный «Тест» во время Y1. При реализации диагностики символ с dp должен сохраняться до следующего нажатия, но не дольше T3; для общего анода включение dp требует нуля в бите 7.

В трёх самостоятельных программах реализованы только заданные процедуры, поэтому таблица не означает наличия в них полного обработчика этих сообщений. KbScan возвращает FFh и для отсутствия контактов, и для нескольких контактов, не различая причины по A; для раздельных сообщений «1.» и «2.» полной системе требуется диагностический счётчик опроса. BufRead безопасно возвращает ноль при пустоте и снимает F_OVF после успешного чтения. Show выводит коды 0–10 без dp и гасит индикатор для прочих кодов. Формирование диагностического dp относится к алгоритму полной системы, а не к проверяемому интерфейсу Show.

Сигнал Y_2 выдаётся 8-разрядным кодом в выходной регистр (адрес 0C000h), строб — на выводе P1.5; сигнал Y_1 — на выводе P1.4.

В 16-разрядном режиме (режим 1) таймер 0 перезагружается в обработчике прерывания; задержка входа в прерывание (несколько машинных циклов на тик) компенсируется поправкой к загружаемому значению или учитывается как погрешность длительности. В режиме 2 (8 разрядов с автоперезагрузкой) перезагрузку выполняет сам таймер, а тик набирается программным счётчиком переполнений.

ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ

Таблица 5.3

Сокращения

Сокращение	**Расшифровка**
МК	микроконтроллер
МПС	микропроцессорная система
ОЗУ	оперативное запоминающее устройство
ПЗУ	постоянное запоминающее устройство
ЕСПД	единая система программной документации
ТЗ	техническое задание
CS	Chip Select, выбор микросхемы
IRQ	запрос прерывания
SP	указатель стека
DPTR	16-разрядный регистр-указатель данных
dp	десятичная точка индикатора

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Изм.	Номера листов (страниц)				Всего листов (страниц) в докум.	№ докум.	Входящий № сопроводит. докум. и дата	Подп.	Дата
	изменённых	заменённых	новых	аннулиро-ванных					

ПРИЛОЖЕНИЕ А

Листинги программ

Программа 1 — файл «Сапелин АС-код-1»

```
; Сапелин А.С., А-12-23, 18, v1
$INCLUDE (vars.inc)
DSEG AT 30h
CSEG AT 0h
        sjmp START
org 03h
        call KbScan ; %proc%
        reti
org 0bh
        nop
        reti
org 13h
        nop
        reti
org 1bh
        nop
        reti
org 23h
        nop
        reti
org 2bh
START:
        mov SP, #07h            ; стек размещается ниже переменных и битовых флагов
        mov PSW, #0             ; используется нулевой банк регистров
        setb PIN_CSEN           ; разрешить выбор устройств внешней шины
        mov DPTR, #ADR_KB
        clr A
        movx @DPTR, A           ; все столбцы готовы обнаружить новое нажатие
        setb IT0                ; однократный запрос по спаду INT0
        clr IE0                 ; исключить запрос от начальной установки столбцов
        setb EX0
        setb EA
        jmp $ ; %stop%

; Назначение: KbScan проверяет все контакты матрицы и считает нажатые клавиши.
; Вход: строки и столбцы клавиатуры, подключённой к ADR_KB.
; Выход: A — код 0–11 единственной клавиши либо 0FFh при ином числе нажатий.
; Портит: A; остальные используемые регистры сохраняются.
KbScan:
        push PSW                ; флаги прерванной программы не должны зависеть от опроса
        push DPL                ; основная программа может обращаться к другой ячейке шины
        push DPH
        push AR0                ; сохранить прежнюю рабочую маску вызывающей программы
        push AR1                ; индекс клавиши нужен только на время опроса
        push AR2                ; счётчик контактов не меняет регистры основной программы
        push AR3                ; временный код клавиши сохраняется отдельно от результата A
        push AR4
        push AR5
        mov R0, #0FEh           ; бегущий ноль начинает со столбца Col1
        mov R1, #0              ; индекс первого контакта столбца в таблице
        mov R2, #0              ; пока ни одна клавиша не обнаружена
        mov R3, #0FFh
KbColumn:
        mov DPTR, #ADR_KB
        mov A, R0
        movx @DPTR, A           ; выбрать один столбец
        movx A, @DPTR
        cpl A
        anl A, #(1 SHL KB_ROWS)-1 ; оставить только реально подключённые строки
        mov R4, A
        mov R5, #KB_ROWS        ; проверить каждую подключённую строку выбранного столбца
KbRow:
        mov A, R4
        rrc A                   ; очередная строка попадает в перенос
        mov R4, A
        jnc KbNextRow
        inc R2                  ; считаются все замкнутые контакты
        mov DPTR, #KeyMap
```

```
        mov A, R1
        movc A, @A+DPTR
        mov R3, A
KbNextRow:
        inc R1                  ; перейти к коду следующего контакта в таблице
        djnz R5, KbRow
        mov A, R0
        rl A
        mov R0, A               ; перенести активный ноль на следующий столбец
        cjne R1, #(KB_COLS*KB_ROWS), KbColumn
        mov DPTR, #ADR_KB
        clr A
        movx @DPTR, A           ; вернуть все столбцы в состояние ожидания
        clr IE0                 ; убрать фронты, возникшие во время сканирования
        mov A, R2
        cjne A, #1, KbInvalid
        mov A, R3
        sjmp KbDone
KbInvalid:
        mov A, #0FFh
KbDone:
        pop AR5
        pop AR4
        pop AR3
        pop AR2
        pop AR1
        pop AR0
        pop DPH
        pop DPL
        pop PSW
        ret
KeyMap:
        db 1,4,7,2,5,8,3,6,9,10,11,0
END
```

Программа 2 — файл «Сапелин АС-код-2»

```
; Сапелин А.С., А-12-23, 18, v1
$INCLUDE (vars.inc)
DSEG AT 30h
CSEG AT 0h
        sjmp START
org 03h
        nop
        reti
org 0bh
        nop
        reti
org 13h
        nop
        reti
org 1bh
        nop
        reti
org 23h
        nop
        reti
org 2bh
START:
        mov SP, #07h            ; начальный стек по шаблону ТЗ
        mov IE, #0              ; самостоятельная программа не обслуживает IRQ
        setb PIN_CSEN           ; доступ к ОЗУ разрешается дешифратором
        mov HEAD_L, #LOW BUF_START ; следующая запись начнётся с начала кольца
        mov HEAD_H, #HIGH BUF_START
        mov TAIL_L, #LOW BUF_START ; чтение и запись пока имеют общий адрес
        mov TAIL_H, #HIGH BUF_START
        setb F_EMPTY            ; в памяти ещё нет действительных отсчётов
        clr F_OVF               ; исходное состояние не является переполнением
        call BufRead ; %proc%
        jmp $ ; %stop%

; Назначение: BufRead извлекает самый старый отсчёт из кольцевого буфера.
; Вход: HEAD, TAIL и флаги из vars.inc; данные во внешнем ОЗУ.
; Выход: A — прочитанный байт, при пустом буфере A = 0 без изменения указателей и флагов.
; Выход: после чтения TAIL передвинут, F_OVF = 0, F_EMPTY отражает опустошение.
```

```
; Портит: A, DPTR, флаги PSW; B сохраняется.
BufRead:
        jnb F_EMPTY, ReadData    ; пустой буфер нельзя читать и передвигать
        clr A
        ret
ReadData:
        push B                   ; временное хранение отсчёта не меняет B вызывающей программы
        mov DPL, TAIL_L
        mov DPH, TAIL_H          ; адрес самого старого отсчёта
        movx A, @DPTR
        mov B, A                 ; сохранить результат на время проверки границ
        inc DPTR                 ; предварительно перейти к следующему отсчёту
        mov A, DPL
        cjne A, #LOW BUF_END, ReadStore
        mov A, DPH
        cjne A, #HIGH BUF_END, ReadStore
        mov DPTR, #BUF_START     ; конец кольца переходит в его начало
ReadStore:
        mov TAIL_L, DPL
        mov TAIL_H, DPH          ; зафиксировать новый адрес чтения
        clr F_OVF                ; прочитанный отсчёт освободил место
        clr F_EMPTY
        mov A, DPL
        cjne A, HEAD_L, ReadDone ; разные указатели означают оставшиеся данные
        mov A, DPH
        cjne A, HEAD_H, ReadDone
        setb F_EMPTY             ; хвост догнал голову после последнего отсчёта
ReadDone:
        mov A, B                 ; вернуть именно отсчёт, а не байт адреса
        pop B
        ret
END
```

Программа 3 — файл «Сапелин АС-код-3»

```
; Сапелин А.С., А-12-23, 18, v1
$INCLUDE (vars.inc)
DSEG AT 30h
TicksLo: DS 1
TicksHi: DS 1
CSEG AT 0h
        sjmp START
org 03h
        nop
        reti
org 0bh
        ljmp T0Isr
org 13h
        nop
        reti
org 1bh
        nop
        reti
org 23h
        nop
        reti
org 2bh
START:
        mov SP, #07h             ; переменные 30h и выше не пересекаются со стеком
        mov IE, #0
        setb PIN_CSEN            ; разрешить запись в регистр индикации
        clr TR0
        clr TF0                  ; удалить прежний запрос переполнения
        mov TMOD, #01h           ; Timer0 считает 16-разрядные интервалы
        mov TicksLo, #0
        mov TicksHi, #0          ; до первого вызова отсчёт не ведётся
        mov DPTR, #ADR_IND
        mov A, #IND_OFF
        movx @DPTR, A            ; при включении все сегменты погашены
        setb ET0
        setb EA
        mov A, #5                ; демонстрационный вывод цифры 5
        call Show ; %proc%
        jmp $ ; %stop%
```

```
; Назначение: Show выводит символ с автоматическим гашением через T3.
; Вход: A — код 0–9 или 10 для E; остальные коды гасят индикатор.
; Выход: символ в регистре ADR_IND, заново запущенный отсчёт T3 либо остановка.
; Портит: A, DPTR, PSW; обновляет TicksLo, TicksHi и Timer0.
Show:
        clr ET0                 ; исключить IRQ со старым счётчиком во время замены символа
        clr TR0
        clr TF0                 ; повторный вызов отменяет предыдущий срок
        mov TicksLo, #0
        mov TicksHi, #0
        cjne A, #11, ShowRange
        sjmp ShowOff
ShowRange:
        jnc ShowOff             ; код за границей таблицы означает выключение
        mov DPTR, #Segments
        movc A, @A+DPTR          ; код символа преобразуется в набор сегментов
        xrl A, #IND_OFF          ; общий анод требует активного нуля
        mov DPTR, #ADR_IND
        movx @DPTR, A           ; регистр удержит изображение после освобождения шины
        mov TicksLo, #LOW T3_TICKS
        mov TicksHi, #HIGH T3_TICKS ; число тиков берётся из параметров варианта
        mov TH0, #TICK_H
        mov TL0, #TICK_L         ; первый тик начинается с полной длительности
        setb ET0
        setb TR0                ; процедура возвращается, время отсчитывается в IRQ
        ret
ShowOff:
        mov DPTR, #ADR_IND
        mov A, #IND_OFF
        movx @DPTR, A           ; неверный код сразу выключает все сегменты
        setb ET0
        ret

; Назначение: T0Isr отсчитывает тики и выключает индикатор в конце T3.
; Вход: TicksLo:TicksHi и текущее состояние Timer0.
; Выход: счётчик уменьшен; при нуле Timer0 остановлен и индикатор погашен.
; Портит: Timer0 и счётчик; A, PSW и DPTR восстанавливаются перед RETI.
T0Isr:
        push ACC                ; обработчик может прервать работу с любым символом
        push PSW
        push DPL
        push DPH                ; сохранить адрес устройства, использованный основной программой
        clr TR0
        mov A, TL0              ; учесть время между переполнением и входом в обработчик
        add A, #(TICK_L+7)      ; компенсировать семь циклов остановленного таймера
        mov TL0, A
        mov A, TH0
        addc A, #TICK_H
        mov TH0, A
        setb TR0                ; задержка обслуживания не накапливается в каждом тике
        mov A, TicksLo
        jnz TickDec
        dec TicksHi             ; заём при уменьшении 16-разрядного счётчика
TickDec:
        dec TicksLo
        mov A, TicksLo
        orl A, TicksHi
        jnz TickDone            ; ненулевой остаток сохраняет текущую индикацию
        clr TR0
        mov DPTR, #ADR_IND
        mov A, #IND_OFF
        movx @DPTR, A           ; срок T3 истёк
TickDone:
        pop DPH
        pop DPL
        pop PSW
        pop ACC
        reti
Segments:
        db 03Fh,006h,05Bh,04Fh,066h,06Dh,07Dh,007h,07Fh,06Fh,079h
END
```